```
MyDelegate example1 = GetMonth;                ❷
MyDelegate example2 = PointlessAddition;       ❷
Console.WriteLine(example1(DateTime.Now));     ❸
Console.WriteLine(example2(DateTime.Now));     ❸
```

❶ Both methods take a DateTime parameter and return an integer.

❷ Assign both methods to delegate instances.

❸ Invoke the methods via the delegate instances.

You can assign an inline anonymous method to a delegate:

```
MyDelegate example3 = delegate(DateTime dt) {
➥ return dt.Now.AddYears(-100).Year; };
Console.WriteLine(example3(DateTime.Now));
```

More commonly, you will see anonymous inline methods written as lambda expressions, where the data type of the method parameters are inferred:

```
MyDelegate example4 = (dt) => { return dt.Now.AddYears(-100).Year; };
Console.WriteLine(example4(DateTime.Now));
```

So any method that takes an `HttpContext` as a parameter and returns a task can be used as middleware.

As you saw earlier, middleware is added to the pipeline via the `WebApplication`. Typically, middleware is created as separate classes that are registered via extension methods, but it is also possible to add `RequestDelegate`s directly to the pipeline. Listing 2.8 shows a simple method that takes an `HttpContext` as a parameter and returns a `Task`, which means it meets the `RequestDelegate` type specification. You can add the method to Program.cs if you want to try this example out. You will also need to add a `using` directive to the `Startup` class to bring `Microsoft.AspNetCore.Http` into scope.

Listing 2.8 `RequestDelegate` takes `HttpContext` as parameter and returns `Task`